

Récurtivité

BUT Informatique - S5 - Algorithmique

1. Arithmétique récursive

1.1. Somme

Écrire un algorithme récursif de calcul de la *somme* de deux entiers *naturels* `a` et `b` en supposant que les seules opérations de base dont vous disposez sont

- l'ajout de 1 à un entier `x` : `x+1`
- le retrait de 1 à un entier `x` : `x-1`
- et la comparaison à 0 d'un entier `x` : `a==0`.

Pour ce faire, vous écrirez les méthodes correspondantes suivantes:

```
static public int inc(int x);
static public int dec(int x);
static public boolean isZero(int x);
static public int add(int a, int b);
```

où `add` ne peut donc utiliser aucun opérateur arithmétique en dehors de fonctions `inc` et `dec`.

Code [solution](#)

La solution découle des équations suivantes:

$$\begin{aligned}a + 0 &= 0 \\ a + (b + 1) &= (a + 1) + b\end{aligned}$$

```
1: static public int inc(int x){
2:     return x+1;
3: }
4:
5: static public int dec(int x){
6:     return x-1;
7: }
8:
9: static public boolean isZero(int x){
10:    return (x==0);
11: }
12:
13: static public int add(int a, int b){
14:     if (isZero(b)) return a;
15:     else return add(inc(a),dec(b));
16: }
```

1.2. Produit

Écrire un algorithme récursif de calcul du *produit* de deux entiers *naturels* `a` et `b` en supposant que les seules opérations de base dont vous disposez sont

- la somme de deux entiers `a` et `b` : `a+b`
- le retrait de 1 à un entier `a` : `a-1`
- et la comparaison à 0 d'un entier `a` : `a==0`.

Vous devez donc implémenter la méthode:

```
static public int mult(int a, int b);
```

Code [solution](#)

La solution découle des équations suivantes:

$$\begin{aligned}a * 0 &= 0 \\ a * (b + 1) &= a + (a * b)\end{aligned}$$

```
1: static public int mult(int a, int b){
2:     if (b == 0) return 0;
3:     else return add(a, mult(a, dec(b)));
4: }
```

1.3. Puissance entière

Écrire un algorithme récursif de calcul de la puissance `n`-ième (`n` entier > 0) d'un nombre réel `a` en supposant que les seules opérations de base dont vous disposez sont

- le produit de deux réels `a` et `b` : `a*b`
- le retrait de 1 à un entier `a` : `a-1`
- et la comparaison à 0 d'un entier `a` : `a==0`.

Vous devez donc implémenter la méthode:

```
static public int puis(int a, int b);
```

Code [solution](#)

La solution découle des équations suivantes:

$$\begin{aligned}a^0 &= 1 \\ a^{b+1} &= a * (a^b)\end{aligned}$$

```
1: static public int puis(int a, int b){
2:     if (isZero(b)) return 1;
3:     else return mult(a,puis(a,dec(b)));
4: }
```

2. Fibonacci récursif vs. itératif

La suite de Fibonacci, pour $n \geq 0$ est définie par l'équation de récurrence suivante:

$$F(n) = \begin{cases} n & \text{si } n \leq 1 \\ F(n-1) + F(n-2) & \text{sinon} \end{cases}$$

2.1. Question 1

Implémentez une méthode récursive

```
public static int fibo_rec_naif(int n);
```

qui calcule le terme de rang n en suivant explicitement la définition précédente. Vous inclurez un compteur qui calcule le nombre d'additions réalisées par cette méthode.

2.2. Question 2

Implémentez la même fonction mais de manière itérative:

```
public static int fibo_iter(int n);
```

Utilisez également un compteur pour compter le nombre d'additions réalisées (on ne tiendra pas compte des additions effectuées sur l'indice de boucle pour réaliser les itérations). Comparez les performances des deux méthodes. Que constatez-vous ? Pourquoi la méthode récursive est-elle aussi inefficace ?

2.3. Question 3

Implémentez une méthode récursive

```
public static int fibo_rec(int n);
```

qui calcule le n -ième terme de la suite de Fibonacci avec le même nombre d'additions que la méthode itérative.

[solution](#)

```
import java.util.Scanner;

public class Main {
    private static class Pair {
        public int x;
        public int y;

        public Pair(int x, int y){
            this.x = x;
            this.y = y;
        }
    }

    private static int cpt = 0;

    public static int fibo_rec_naif(int n){
        if (n<2) return n;
        else {
            cpt++;
            return fibo_rec_naif(n-1) + fibo_rec_naif(n-2);
        }
    }

    public static int fibo_iter(int n){
        int n1=0, n2=1, tmp;
        for (int i = 2; i<=n; i++){
            tmp=n1;
            n1=n2;
            n2+=tmp;
            cpt++;
        }
        if (n==0) return 0;
        else return n2;
    }

    public static int fibo_rec(int n){
        return fibo_rec_aux(n).x;
    }

    public static Pair fibo_rec_aux(int n){
        if (n==0) return new Pair(0,1);
        else {
            Pair p = fibo_rec_aux(n - 1);
            cpt++;
            return new Pair(p.y, p.x + p.y);
        }
    }

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        int n = 0;
        while (n>=0) {
            System.out.print("Rang: ");
            n = scanner.nextInt();
            cpt = 0;
            System.out.println("fib_rec_naif(" + n + ") = " + fibo_rec_naif(n) + " ( " + cpt + " sommes )");
            cpt = 0;
            System.out.println("fib_iter(" + n + ") = " + fibo_iter(n) + " ( " + cpt + " sommes )");
            cpt = 0;
            System.out.println("fib_rec(" + n + ") = " + fibo_rec(n) + " ( " + cpt + " sommes )");
        }
    }
}
```

3. Permutations

On appelle permutation d'une chaîne de caractères s toute chaîne de même longueur que s contenant les mêmes caractères que s . Par exemple, la chaîne *eadbc* est une permutation de la chaîne *abcde*. Réalisez une fonction récursive qui construit la liste de toutes les permutations possibles d'une chaîne s . On supposera, pour ne pas avoir à gérer les éventuelles répétitions de mots, que toutes les lettres du mot d'entrée sont distinctes les unes des autres. Vous aurez très probablement à implémenter une fonction auxiliaire qui sera également récursive.

Code [solution](#)

La méthode `insert(char c, String m, int i)` renvoie la liste des mots dans lesquels le caractère `c` a été inséré à chacune des positions `p ≥ i` du mot `m`. Par exemple `insert('a', "bbb", 1)` renvoie la liste `["babb", "bbab", "bbba"]` et `insert('a', "bbb", 3)` renvoie la liste `["bbba"]`.

```
1: public static ArrayList<String> insert(char c, String mot, int i){
2:     if (i>mot.length()) return new ArrayList<>();
3:     else {
4:         ArrayList<String> res = insert(c, mot, i+1);
5:         res.add(mot.substring(0,i) + c + mot.substring(i,mot.length()));
6:         return res;
7:     }
8: }
```

```
1: static public ArrayList<String> permutations(String mot){
2:     int lg = mot.length();
3:     char c = mot.charAt(0);
4:     ArrayList<String> permuts = new ArrayList<>();
5:     if (lg <= 1) {
6:         permuts.add(new String(mot));
7:     } else {
8:         for (String m : permutations(mot.substring(1, lg)))
9:             permuts.addAll(insert(c, m, 0));
10:    }
11:    return permuts;
12: }
```

Pour une solution optimale, voir l'[algorithme de Heap](#).

Auteur: Cédric Lhoussaine